

similar-sets2 - Project Guide

Changes, programs, and what to use for what • Master thesis: cache-friendly block skip list for the set-trie • 2026-07-10

1. What the data structure is

cskiplist is a **block skip list**: keys (with value pointers) are stored in sorted arrays called *blocks*, and a skip list is built *on top of the blocks*. A search first skips from block to block in $O(\log n)$ and then searches inside one cache-resident block. This is exactly the design from the meetings with prof. Savnik, and it will later become the node structure of his set-trie for set-similarity search (the adapter is `connector_csl.c`).

Search inside a block has three regimes:

Block state	Method	Why
≤ 32 items, sorted	SSE2 SIMD linear scan (4 keys per instruction)	No branch mispredictions, sequential access - fastest for tiny blocks
> 32 items, sorted	Branchless binary search (CMOV lower bound)	$O(\log B)$ comparisons without pipeline flushes (Khuong & Morin, sec. 3.1)
Eytzinger enabled	Heap-ordered (BFS) layout + branchless descent + prefetch	Cache-friendly forward-only access (Khuong & Morin, sec. 4) - the thesis topic

Key reference paper: P.-V. Khuong, P. Morin, *Array Layouts for Comparison-Based Searching* (arxiv.org/abs/1509.05053).

2. Changes made (2026-07-10)

2.1 Critical bug fix: SIMD search corrupted random inserts

The last commit (3627f66) introduced the SIMD scan for small blocks. Its early-exit only checked the *first* key of each 4-key group, so when a missing key fell *inside* a group, the reported insertion position was wrong by up to 3 slots. Every random-order insert could therefore place a key out of order, silently corrupting the block (confirmed: 92 of 1000 keys became unfindable). **Any experiment run on the old code would have been invalid.** Fixed in `blk_binary_search()` with a second SIMD compare (`_mm_cmpgt_epi32`) that finds the exact lower bound inside the group.

2.2 Insertion now maintains the skip pointers (professor's requirement)

Before: `csl_insert` found the target block with a *linear scan* and set `sl->level = 0` after every split - the structure degraded to a plain linked list until you manually called `csl_rebuild_skips()`. The professor explicitly asked for insertion that updates the skip lists ("the algorithm is not so trivial").

Now: every new block receives a **probabilistic tower height** (geometric, $p = 1/2$, like a classic skip list whose nodes are whole blocks) and is **spliced into all of its levels at creation time**; emptied blocks are unspliced on delete. Searches stay $O(\log n)$ at all times with **no rebuild needed**. `csl_rebuild_skips()` still exists as an optional optimizer that produces perfectly balanced deterministic skips after a bulk load.

2.3 Other fixes and improvements

Problem	Fix / effect
<code>csl_append</code> walked the whole chain to find the last block - bulk loading was quadratic	Tail pointer in the struct: 1M sorted appends now run at ~8 ns/op
Duplicate key appended when the tail block was full (key equal to current maximum)	Update is checked before allocating a new block
Descending-order inserts created one new block per key (never filled blocks)	Key smaller than all blocks now goes into the first block (splits normally)

Problem	Fix / effect
Memory leak: emptied blocks were freed without their items[] and next[] arrays	All three allocations freed
Requested block capacity silently clamped to [4, 2048] - would sabotage the block-size sweep (professor wants 5 ... few thousand)	csl_create_with_block_cap() honors the request exactly; the TLB heuristic remains available as csl_tlb_aware_block_cap_hint()
Classic skip list baseline used fixed 32-pointer nodes (~272 B/node) - unfair memory/cache comparison	skiplist.c now allocates variable-height nodes (flexible array member)
Out-of-order csl_append returned an error when the tail block was full	Falls back to csl_insert transparently

2.4 Files touched

File	Status	What
src/C/cskiplist.c / .h	modified	SIMD fix, incremental skip maintenance, tail pointer, edge-case fixes
src/C/skiplist.c / .h	modified	variable-height nodes (fair baseline)
src/C/test-experiment.c	NEW	unified experiment driver (section 4)
src/C/run-experiments.ps1	NEW	runs the whole experiment matrix
src/C/EXPERIMENTS.md	NEW	experiment documentation + thesis graph guide
src/C/Makefile	modified	new 'experiment' and 'clean' targets; fixed missing -lpsapi on two targets
src/C/results/	NEW	first experiment data (CSV, pandas-ready)

Nothing is committed yet - review with `git diff` and commit when happy. All existing tests pass, and `testproc` (cskiplist-backed set-trie) produces results identical to `testproc-base` (original array connector).

3. Programs: what runs what

Everything lives in `src/c`. Build all binaries with `make all` (MinGW gcc; Git Bash or PowerShell). `make clean` removes binaries and `.o` files.

3.1 The main tool

Program	What it does
<code>experiment.exe</code>	THE tool for the thesis. One binary compares 6 structures on the identical query stream, sweeps block sizes at runtime (no recompiling), verifies all structures return identical results, and writes CSVs with the parameters encoded in the file name. Built by 'make experiment'. Full usage in section 4.
<code>run-experiments.ps 1</code>	Runs the whole default matrix: size sweep 10K-4M across cache levels, block caps 8-4096, tiny-set runs ($n = 8/32/256$), insert benchmark; merges everything into <code>results/all-results.csv</code> . Use <code>-Quick</code> for a fast reduced pass.

3.2 Correctness tests (run these after touching `cskiplist.c`)

Program	What it checks	How to run
<code>cskiptest.exe</code>	basic insert / search smoke test	<code>./cskiptest</code>
<code>cskiptest-enh.exe</code>	random insert/delete/search mix, runtime block caps, per-trie-level caps	<code>./cskiptest-enh</code>
<code>cskiptest-million.exe</code>	large-scale: sequential + random insert, delete, iterate, update, stress	<code>./cskiptest-million small medium large xlarge</code>
<code>eyttest.exe</code>	Eytzinger layout: conversion round-trip, search, seek, iteration, insert/delete while enabled; then cache-latency benchmarks	<code>./eyttest [N] [queries]</code>
<code>skiptest.exe</code>	classic skip list baseline smoke test	<code>./skiptest</code>
<code>askiptest.exe</code>	adaptive structure (sorted array below a threshold, block skip list above)	<code>./askiptest</code>
<code>testproc / testproc-base</code>	end-to-end set-trie similarity search; <code>testproc</code> uses the <code>cskiplist</code> connector, <code>testproc-base</code> the original array connector - their outputs must be identical	<code>./testproc sample.txt.mapd.sorted sample.txt.mapd.test.sorted 2</code>

3.3 Older / specialised benchmarks (kept, but mostly superseded)

Program	What it does	Still useful for
<code>skipbench.exe</code>	<code>cskiplist</code> vs an embedded reference skip list (begeekmyfriend)	cross-checking the skiplist baseline; 'experiment' covers this now
<code>cachebench.exe + run-cache-bench.ps1</code>	block-size sweep by recompiling with <code>-DCSL_BLOCK_CAP</code> per size	superseded: 'experiment -b' sweeps caps at runtime in one binary
<code>simdbench.exe</code>	SIMD vs scalar block-search microbenchmark	isolating the SIMD effect only
<code>branchless.exe</code>	branchy vs branchless search microbenchmark	isolating the branchless effect only
<code>set2 / hat</code>	professor's original set-trie programs (untouched)	reference behaviour / regression comparison

4. The experiment driver: usage

Build: `make experiment` or `gcc -O3 -msse2 -o experiment cskiplist.c skiplist.c test-experiment.c -lpapi`

4.1 Structures compared (identical query sequence for all)

Name in CSV	Structure	Role in the thesis
array	plain sorted array, classic branchy binary search	textbook baseline
array-bl	plain sorted array, branchless binary search	shows the branchless effect alone
array-eyt	whole array in Eytzinger layout, branchless	answers the professor's 'maybe a plain array is faster' question
skiplist	classic probabilistic skip list (one node per key)	the structure the thesis must beat
cs1	block skip list, sorted blocks (swept over block caps)	your structure, baseline layout
cs1-eyt	block skip list, Eytzinger blocks (swept over caps)	your structure, thesis layout

4.2 Options

Option	Meaning	Default
-m search insert	workload: query benchmark, or random-order insert benchmark	search
-n N	number of keys in the structure	1000000
-q Q	number of search queries	500000
-b c1,c2,...	block capacities to sweep (runtime, no recompiling)	16...2048
-r reps	repetitions per configuration (rows are written per rep)	3
-H pct	percentage of queries that hit an existing key	50
-d uniform dense	key distribution (uniform = random half of a 2N universe)	uniform
-f file	load keys from a file of whitespace-separated integers (use for the professor's data sets); misses are generated automatically	-
-s seed	RNG seed - keep fixed for reproducible runs	42
-o dir	output directory for CSVs	results

4.3 Output

One CSV per run, parameters in the file name (the professor's advice on organizing data), one row per structure × block cap × repetition:

```
results/exp_search_uniform_n1000000_q500000_hit50_seed42.csv
```

Column	Meaning
structure, layout, block_cap	which variant (block_cap = 0 means not applicable)
n, q, dist, hit_pct, seed, rep	experiment parameters; rep = repetition index (raw rows, average yourself)
build_ms	bulk-load time (sorted appends; in insert mode: total insert time)
prep_ms	cs1_rebuild_skips + Eytzinger conversion time
search_ns	average wall-clock nanoseconds per query in this repetition
insert_ns	ns per random-order insert (insert mode only)
hits / expected_hits	must be equal - correctness cross-check; the run fails loudly otherwise
mem_bytes, bytes_per_key	exact structural memory (counted by walking the structure, not RSS)

The [VERIFY] line at the end says whether all six structures returned identical results on every query (the professor's "results have to be identical" requirement). Exit code is non-zero on mismatch.

5. What should I use for what

I want to ...	Do this
check I did not break anything after editing cskiplist.c	make all, then run ./cskiptest, ./cskiptest-enh, ./cskiptest-million small, ./eyttest; finally compare ./testproc vs ./testproc-base on the sample data (results must be identical)
produce ALL data for the thesis in one go	.\run-experiments.ps1 (PowerShell, in src/C). Takes a few minutes; everything lands in results/, merged into all-results.csv
find the optimal block size for a given data-set size	./experiment -n 1000000 -b 8,16,32,64,128,256,512,1024,2048,4096 - then plot search_ns vs block_cap
show the Eytzinger effect (the thesis core)	compare csl vs csl-eyt rows from the same run; the gap grows with n and block size
run on the professor's data sets (when he sends them)	./experiment -f dataset.txt - one file of whitespace-separated integers; n is taken from the file, misses are generated automatically
measure hit vs miss latency separately	./experiment -H 100 ... and ./experiment -H 0 ... with the same seed, then compare search_ns
benchmark random-order insertion (incremental skip maintenance)	./experiment -m insert -n 200000 -b 32,128,512,2048
simulate tiny set-trie nodes (3-10 keys per node)	./experiment -n 8 -b 4,8,16 (also n=32, 256) - at this size the plain array wins; that is what askiplist (adaptive threshold) is for
analyze the CSVs	df = pd.read_csv("results/all-results.csv") then group by structure/layout/block_cap/n and average search_ns over rep (see EXPERIMENTS.md for a starter snippet)
pick block sizes per set-trie level in the real integration	csl_create_for_level(level) - large blocks at the root, small at the leaves

6. First results (this machine, gcc -O3, 500K queries, 50% hits, avg of 3 reps)

Search latency in ns per query. Lower is better.

Baselines:

n	array (branchy)	array-bl (branchless)	array-eyt (Eytzinger)	classic skiplist
10 000	94	18	36	147
100 000	171	46	70	273
1 000 000	334	153	134	865
4 000 000	393	258	178	1333

Block skip list (best block cap per size, cap in parentheses):

n	csl sorted blocks	csl Eytzinger blocks	vs classic skiplist
10 000	55 (2048)	51 (2048)	2.9x faster
100 000	133 (2048)	126 (2048)	2.2x faster
1 000 000	373 (4096)	311 (4096)	2.8x faster
4 000 000	499 (128)	354 (4096)	3.8x faster

Memory and inserts:

Metric	Block skip list	Classic skip list
memory per key (n = 1M)	8.0 - 8.3 B/key	17.3 B/key
random-order insert (n = 200K)	301 ns (cap 128)	381 ns

What the numbers mean for the thesis

- **Block skip list beats the classic skip list 2.2-3.8x on search, at half the memory**, and even wins on random inserts. This is the headline result.
- **Eytzinger blocks pay off once data exceeds the caches and blocks are large** (4M keys, cap 4096: 354 vs 530 ns for sorted blocks) - exactly the Khuong-Morin effect the professor wants demonstrated.
- **The optimal block size is 2048-4096 items = 16-32 KB, about the L1 data cache** - this confirms the professor's L1 hypothesis with data.
- **A static sorted array in Eytzinger layout is still ~2x faster than any skip list** (178 vs 354 ns at 4M) - the professor's hunch was right, BUT arrays cannot absorb inserts; the skip list is the price of being dynamic. Good discussion chapter.
- **For tiny nodes (n = 8-32) a plain branchless array wins at ~5 ns vs ~20-40 ns for any skip list** - which justifies the adaptive askiplist (array below a threshold) for set-trie leaves.

7. Requirements checklist and next steps

Professor's requirement	Status
Blocks + skip list on top of blocks	DONE
Eytzinger layout inside blocks, branchless search	DONE (csl_set_eytzipger)
SIMD / vectorized search (simple case only)	DONE (SSE2 scan for small blocks)
Block size as a runtime parameter stored in the structure	DONE
Insertion maintains the skip lists	DONE (incremental, probabilistic towers)
Block-size sweep experiments (5 ... few thousand)	DONE (experiment -b, runtime)
Random queries, ~50% hits / 50% misses	DONE (-H, default 50)
Compare vs classic skip list and plain sorted array	DONE (6 structures, verified identical results)
Results in files with parameters in the file name	DONE
Key + value (pointer) for the set-trie	DONE (csl_kv, connector_csl.c)
Run on the professor's real set data sets	WAITING for the data (-f is ready)
TLB-aware block counting (radix-join trick)	DEFERRED - he said to postpone
Manual vectorization beyond the simple case	DEFERRED - he said only if trivial

Suggested next actions:

- Review the changes (`git diff` in similar-sets2) and commit.
- Ask the professor for his data sets; rerun with `-f`.
- Rerun with `-H 100` and `-H 0` for the hit-vs-miss graph.
- Make the thesis plots from `results/all-results.csv` (graph list in EXPERIMENTS.md, section 4).
- Start writing notes per meeting advice - EXPERIMENTS.md section 6 already logs the history entry for the bug fix and design changes.